Comp-2540: Lab Assignment 3: Stack and Dynamic Array (2026)

Your Marks (For TA Use Only)

3.1.1	3.1.2	3.1.3	3.2.1	3.2.2	Bonus	Total	Marked by

1 Due Date and Submission

The due time is your lab section during the week starting October 27. To be submitted 10 minutes before the end of your lab section. You can also submit in the labs one week earlier or during office hours. Make sure that you add your signature below to reaffirm that you followed Senate Bylaws 31 (click here for the document). Some PDF readers may not support digital signature well. We recommend you to use Acrobat Reader that is free for downloading.

Make sure that you use the **our submission site here**.

Fill and Sign The Form

I, your name Saidjon Nozimboev **, verify that the submitted work is my own work.**

Date 21/10/202 Student Number 110186060 EMail ID noziml

2 Objective

The aim of this assignment is to obtain hands on experience in implementing the stack ADT. To understand an efficient implementation of stack, we use dynamic array to save space. Different from previous lab assignments that focused on time efficiency, this assignment also consider space efficiency, and the balance between the two. From this example, we learn amortized time complexity.

3 Tasks

3.1 Implement the stack ADT using array (2 marks)

3.1.1 Implement the `pop()` operation in the stack (0.5 mark)

Implement a stack class named `Stack2540Array` using array. The starter code is as follows. The instance variables and most operations are provided. You need to implement the `pop` operation. Make sure that your program checks whether the stack is empty in the pop operation. The implementation can be found in the book and in our lecture slides.

```
import java.io.*;
import java.util.*;

public class Stack2540Array {
        int CAPACITY = 128;
        int top;
        String[] stack;

        public Stack2540Array() {
                stack = new String[CAPACITY];
                top = -1;
```

```
        }

        public int size() {             return top + 1; }
        public boolean isEmpty() {              return (top == -1); }

        public String top() {
                if (top == -1)
                        return null;
                return stack[top];
        }

        public void push(String element) {
                top++;
                stack[top] = element;
        }
```

```
public String pop()
{
  if (isEmpty())
     return null;
  String element = stack[top];
  stack[top] = null;
  top--;
  return element;
```

Figure 1: Write the `pop()` operation in the box. Also submit the code in the submission site.

3.1.2 Test Your Stack with the Parentheses Matching Problem (1 marks)

You will test your `Stack2540Array` on an application, i.e., parentheses matching. It takes a string that contains parentheses that are often found in arithmetic expressions, and checks whether the brackets are correct (balanced) or incorrect (unbalanced). Example of valid strings are:

```
final static String[] valid = {
        "( ) ( ( ) ) {( [ ( )  ] ) } ",
        "(3) (3 + (4 - 5) ) {( [ ( )  ] ) } ",
        "((((())){([()])}))",
        "[(5+x)-(y+z)]"
};
```

Examples of invalid strings are:

```
final static String[] invalid={
        ")(())){([()])}",
        "({[])}",
        "("
};
```

You will write a method named `isMatched` like below. It will return true for valid strings, and false for invalid strings. You will use the stack named `Stack2540Array` that is implemented in the first step. The algorithm itself can be the same as in our lecture slides. Your program does not need to check the correctness of the arithmetic operators/operands. What you need to do is to check for balanced brackets.

```
        public static boolean isMatched(String expression) {
                final String opening = "({[";
                final String closing = ")}]";
                Stack2540Array buffer = new Stack2540Array();
                ... ...
                return  ...;
        }
```

3.1.3 Test Memory Usage on the Text File Reversing Problem (0.5 mark)

Test the performance of your Stack implementation on file reversing task. The code below is to reverse a text file using the stack. It has three lines missing, and you need to complete it. Then you submit it on our marking site here. You need to change the stack capacity according to input data size. On average each line has about 11 words. Our site prints out the total computer memory usage by your program when it runs on 5 data sets of different sizes.

The total memory usage from the submission site is 376.69 MB

```
static String[] reverse(String filename) throws Exception{
                Scanner scanner = new Scanner(new File(filename)).useDelimiter("[^a-zA-Z]+");
                Stack2540Array stack = new Stack2540Array();
                while (scanner.hasNext())
                        stack.push(scanner.next().toLowerCase());
```

```
static String[] reverse(String filename) throws Exception {
   Scanner scanner = new Scanner(new File(filename))
         .useDelimiter("[^a-zA-Z]+");

   Stack2540Array stack = new Stack2540Array();

   while (scanner.hasNext()) {
      stack.push(scanner.next().toLowerCase());
   }
   scanner.close();

   String[] rev = new String[stack.size()];
```

Figure 2: Write the `isMatch` method in the box. Also test your code here

```
            String[] rev = new String[stack.size()];
            ...
            ...
            ...
            return rev;
      }
```

3.2 Dynamic Array (2 marks)

To save computer memory, we need to use dynamic array. In the first version of our stack implementation, the instance variable has a fixed capacity, which is 128. There will be an error when the stack is bigger than 128. For bigger data, you increased the array size. But that very long array could be wasted for smaller data.

3.2.1 Array Resizing (1 mark)

In order to solve this dilemma, we start with a small array, and increase the capacity only when it is needed. You will modify the `push` operation by resizing the array when it overflows. And you also need to implement the `resize(...)` method. The details of the code are in the slides. Submit your dynamic stack in the submission site here. You can use the doubling strategy in resizing.

```
public void resize(int newCapacity)
{
   String[] newStack = new String[newCapacity];
   for (int i = 0; i <= top; i++) {
      newStack[i] = stack[i];
   }
   stack = newStack;
   CAPACITY = newCapacity;
```

Figure 3: Write the `resize(...)` operation in the box. Also test your code on our submission site here

To make sure that your resizing is correct, you will test your `Stack2540ArrayDynamic` on the reversing task in our site. The reverse method is called `reverseDynamic`, and it will use the dynamic version of the stack implementation. The total space used by dynamic array is now 236.3 MB

3.2.2 Comparison of doubling strategy and constant incremental approach (1 mark)

There are two approaches to array resizing, i.e., doubling and increasing by a constant. Run `reverseDynamic` program on two versions of Stack2540ArrayDynamic. In one version, array is resized by doubling (CAPACIT*2). In another version, it is resized by adding a constant 128 (CAPACITY+128). We shall see that two strategies causes very different performance. In theory, suppose that the data size (the number of words in the text file) is n. The time complexity of `reverseDynamic` program when doubling strategy is used is O(n) in big O notation. The time complexity of `reverseDynamic` when constant addition is used is O(n^2). A brief explanation of complexity of the doubling strategy can be explained by the total copying cost, which is calculated by the following formula:

In the doubling strategy, the array capacity doubles whenever the array becomes full. The total copying cost is bounded by a geometric series:
1 + 2 + 4 + 8 + ... + 2^k < 2n
Therefore, the total copying cost is O(n), and the time complexity of reverseDynamic is O(n).
In the constant-increment strategy, the array capacity increases by 128 each time. The array must be copied repeatedly as its size increases. The total copying cost is approximately:
128 + 256 + 384 + ... + n = $O(n^2)$

Therefore, the time complexity is $O(n^2)$ for constant-increment resizing, while doubling is more efficient for large files.

4 Bonus (1 mark for each of the top 10 students)

When you run the dynamic array, we print out both memory usage and running time, and the summation of the two. You will receive one bonus mark if you are among the top 10 programs (most efficient in both space and time).